

Software Development Fundamentals (SDF) – I

ODD 2023



Module 02: Data types, operators, and Control Flow

Module 2 Outline

- Data, variables and constants, data types in C programming
- Operators – binary, unary, ternary, operator precedence, operations using different operators in C programming
- Control Flow- if, if-else, while, do-while, for, switch-case in C programming

Lecture Outline

- Data in C
- Variables in C
- Keywords in C
- Constant in C
- Data Types in C

Data

- Data is a piece of information that we used in programming to execute any program.
- Data stores the numbers, words, measurements, observations or just descriptions of things.
- Data are characteristics or information, usually numerical or text.
- These data are used in the programming language to execute the programme.

Variables

- Variables are used to store the data.
- If you declare a variables in the programme means memory will be allocated to that variable when program executes.
- These variable value can be changed during the programme execution.
- But if any variable is declared constant means that variable value can't be change.
- Each variable should be given a unique name that unique name will be used throughout the programme.

- Variable names are just the symbolic representation of a memory location.
- Variables are stored in memory and each variable has a unique address in memory region. For each variable, memory is reserved in the computer.
- It is difficult to remember the unique address, thus for convenience, you can assign a name to the variables.
- In variables, you can store different types of data such as integer, string, float, characters. All these data are of different sizes in computer.
- Thus, computer needs to know what type each variable is – so it can reserve the memory you need .

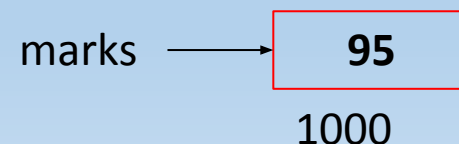
Variable declaration syntax:

```
datatype    variablename;
```

For example: `int marks=95`

Here **marks** is a variable of **int** type and the variable is assigned an values **95**.

Variable marks is stored in memory and variable marks have some memory address and here marks has 1000 memory address. In the memory these variables looks like:



Rules for Variable/Identifiers declaration

- Variable declaration contains only letters, digits and under score characters,
example: marks, student_list
- Variable declaration must be begin with either a letter of the alphabet or an underscore character.
- Variable can not be same as the keywords, example it can not be void, int, extern, char etc.
- Variables names are case sensitive. For example amount, Amount and AMOUNT all three are different variable names.
- Maximum length of the variable can be 31 characters.
- Special characters can't be used in variable declaration for e.g. blank space, #, \$, *, semicolon, comma etc.

Constants in C

- If you want to define a variable whose value can not be change during program execution, you can use the const keyword just before the variable declaration.
- This will create a constant variable .

syntax:

const datatype constant_name

- For example,

```
const double root_two = 1.414;
```

- We have added keyword `const` to make the variable constant. Here `root_two`, is a constant variable. Its value can't be change.

- When you try to change `root_two` from 1.414 to 2.9 it will give an error.

```
root_two= 2.9; // error= constant value can't change.
```

- You can also define a constant using the `#define` pre-processor directive.

Keywords in C

- There are certain words that are reserved in C.
- These words are known as keywords and each keyword have some meaning.
- Keywords are always written in lowercase.
- There are 32 keywords available in C which are given below:

auto	extern	sizeof	break	static
case	for	struct	goto	switch
const	if	typedef	enum	signed
default				
int	union	long	continue	
unsigned	do	register	void	
double	return	volatile	else	
short	while	float	char	

Format Specifiers

- Format specifiers are used with `printf()` statement for printing the data that are stored in variable.
- To print the data stored in variable you need to use the format specifier.
- Format specifiers start with a percentage `%` operator and followed by a special character for identifying the type of data.

Format Specifier	Description
<code>%d</code>	int
<code>%c</code>	char
<code>%f</code>	float
<code>%lf</code>	double
<code>%s</code>	string

Character format specifier : %c

```
#include <stdio.h>
int main()
{
    char ch = 'A';
    printf("%c\n", ch);
    return 0;
}
```

Output:
A

String printing : %s

```
#include <stdio.h>
int main()
{
    char a[] = "JIIT";
    printf("%s\n", a);
    return 0;
}
```

Output: JIIT

Integer format specifier : %d, %i

```
#include <stdio.h>
int main()
{
    int x = 45, y = 90;
    printf("%d\n", x);
    printf("%i\n", x);
    printf("%u\n", x);
    return 0;
}
```

45

45

45

Difference between %d and %i format specifier in C

When you are printing using the printf function, there is no specific difference between the %i and %d format specifiers. But both format specifiers behave differently with scanf function.

The %d format specifier takes the integer number as decimal but the %i format specifier takes the integer number as decimal, hexadecimal or octal type. it means the %i automatically identified the base of the input integer number.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int data1, data2, data3;
```

```
printf("Enter value in decimal format:");
```

```
scanf("%d",&data1);
```

```
printf("data1 = %i\n\n", data1);
```

```
printf("Enter value in hexadecimal format:");
```

```
scanf("%i",&data2);
```

```
printf("data2 = %i\n\n", data2);
```

```
printf("Enter value in octal format:");
```

```
scanf("%i",&data3);
```

```
printf("data2 = %i\n\n", data3);
```

```
return 0;
```

```
}
```

```
Enter value in decimal format:10
```

```
data1 = 10
```

```
Enter value in hexadecimal format:0x10
```

```
data2 = 16
```

```
Enter value in octal format:010
```

```
data2 = 8
```

```
Process returned 0 (0x0)   execution time : 112.604 s
```

```
Press any key to continue.
```

Integer format specifier : %d, %i

```
#include <stdio.h>
int main()
{
    int x = 45, y = 90;
    printf("%d\n", x);
    printf("%i\n", x);
    printf("%u\n", x);
    return 0;
}
```

45

45

45

Floating-point format specifier : %f, %e or %E

```
#include <stdio.h>
int main()
{
    float a = 12.67;
    printf("%f\n", a);
    printf("%e\n", a);
    return 0;
}
```

Output:

12.670000

12.67000+e01

Use of special elements with %f

```
#include <stdio.h>
int main()
{
    float data = 6.276240;
    printf("%f\n", data);
    printf("%0.2f\n", data);
    printf("%0.4f\n", data);
    return 0;
}
```

Output:

6.276240

6.28

6.2762

Format specifiers (octal number): %o

```
#include <stdio.h>
int main()
{
    int data = 65;
    printf("%o\n", data);
    return 0;
}
```

Output: 101

Use of special elements with %s

```
#include <stdio.h>
int main()
{
    char str[] = "Welcometojiit128";
    printf("%s\n", str);
    printf("%3s\n", str);
    printf("%20s\n", str);
    printf("%-20s\n", str);
    printf("%.3s\n", str);
    printf("%.8.3s\n", str);
    return 0;
}
```

Output

```
Welcometojiit128
Welcometojiit128
      Welcometojiit128
Welcometojiit128
Wel
      Wel
```

- A minus(-) sign tells left alignment.
- A number after % specifies the minimum field width to be printed if the characters are less than the size of width the remaining space is filled with space and if it is greater than it printed as it is without truncation.
- A period(.) symbol separate field width with the precision.
- Number after . tells that the maximum number of character to be printed

Data Types in C

- C supports different types of data such as integer, character, float and string etc.
- C provides various types of data-types which allow the programmer to select the appropriate type for the variable to set its value.
- Data types are used to define the type of variable when it declared.
- Data types are used to define the return type of function.
- Data types are used to define the types of parameter expected by a function.
- C supports many data types out of which the basic types are:
void, int, float , double and char.

Derived and User Defined data types

- The derived data types are built using one or more primary data types.
- Example: arrays, functions, structures, pointers.
- In C, a user is allowed to define a data types as per the requirement. These data-types are referred as user-defined data types.
- Examples of user-defined data-types include structure, union, enum.

Four Basic Data Types

- In C, there are 4 basic data types:

char,

int,

Float

Double

- 'char' is used to store any single character
- 'int' is used to store integer value
- 'float' is used for storing single precision floating point number
- 'double' is used for storing double precision floating point number.

Char Data type

- Char type variable store single character.
- Size of char type variable is 1 or 2 byte depends on operating system configuration.
- The range of char is -128 to 127.
- All character have a numeric code associated with them, so while storing a character variable its numeric value gets stored.
- The set of numeric code is called as “ASCII”, American Standard Code for Information Interchange.

ASCII control characters		
00	NULL	(Null character)
01	SOH	(Start of Header)
02	STX	(Start of Text)
03	ETX	(End of Text)
04	EOT	(End of Trans.)
05	ENQ	(Enquiry)
06	ACK	(Acknowledgement)
07	BEL	(Bell)
08	BS	(Backspace)
09	HT	(Horizontal Tab)
10	LF	(Line feed)
11	VT	(Vertical Tab)
12	FF	(Form feed)
13	CR	(Carriage return)
14	SO	(Shift Out)
15	SI	(Shift In)
16	DLE	(Data link escape)
17	DC1	(Device control 1)
18	DC2	(Device control 2)
19	DC3	(Device control 3)
20	DC4	(Device control 4)
21	NAK	(Negative acknowl.)
22	SYN	(Synchronous idle)
23	ETB	(End of trans. block)
24	CAN	(Cancel)
25	EM	(End of medium)
26	SUB	(Substitute)
27	ESC	(Escape)
28	FS	(File separator)
29	GS	(Group separator)
30	RS	(Record separator)
31	US	(Unit separator)
127	DEL	(Delete)

ASCII printable characters		
32	space	
33	!	
34	"	
35	#	
36	\$	
37	%	
38	&	
39	'	
40	(	
41	)	
42	*	
43	+	
44	,	
45	-	
46	.	
47	/	
48	0	
49	1	
50	2	
51	3	
52	4	
53	5	
54	6	
55	7	
56	8	
57	9	
58	:	
59	;	
60	<	
61	=	
62	>	
63	?	
64	@	
65	A	
66	B	
67	C	
68	D	
69	E	
70	F	
71	G	
72	H	
73	I	
74	J	
75	K	
76	L	
77	M	
78	N	
79	O	
80	P	
81	Q	
82	R	
83	S	
84	T	
85	U	
86	V	
87	W	
88	X	
89	Y	
90	Z	
91	[	
92	\	
93	]	
94	^	
95	_	
96	`	
97	a	
98	b	
99	c	
100	d	
101	e	
102	f	
103	g	
104	h	
105	i	
106	j	
107	k	
108	l	
109	m	
110	n	
111	o	
112	p	
113	q	
114	r	
115	s	
116	t	
117	u	
118	v	
119	w	
120	x	
121	y	
122	z	
123	{	
124		
125	}	
126	~	

Extended ASCII characters							
128	Ç	160	á	192	Ł	224	Ó
129	ü	161	í	193	ł	225	ô
130	é	162	ó	194	ƚ	226	Ô
131	â	163	ú	195	ƚ	227	Ò
132	ä	164	ñ	196	—	228	ö
133	à	165	Ñ	197	†	229	Õ
134	å	166	ª	198	ä	230	μ
135	ç	167	º	199	Ä	231	þ
136	ê	168	¿	200	Ł	232	Þ
137	ë	169	®	201	ƚ	233	Ù
138	è	170	™	202	ƚ	234	Ú
139	ï	171	½	203	ƚ	235	Û
140	î	172	¼	204	ƚ	236	Ü
141	ì	173	ı	205	=	237	Ý
142	Ä	174	«	206	≠	238	Ÿ
143	Å	175	»	207	□	239	˘
144	É	176	⋮	208	ø	240	≡
145	æ	177	⋮	209	Ð	241	±
146	Æ	178	⋮	210	Ê	242	≡
147	ô	179	⋮	211	Ë	243	¼
148	ö	180	⋮	212	È	244	¶
149	ò	181	À	213	Ì	245	§
150	û	182	Â	214	Í	246	÷
151	ù	183	Ã	215	Î	247	˙
152	ÿ	184	©	216	Ï	248	˚
153	Ö	185	ƚ	217	ƚ	249	ˆ
154	Ü	186	ƚ	218	ƚ	250	˙
155	ø	187	ƚ	219	■	251	˙
156	£	188	ƚ	220	■	252	˙
157	Ø	189	¢	221	⋮	253	˙
158	×	190	¥	222	⋮	254	■
159	f	191	ſ	223	■	255	nbsp

Char data type

- Character (**char**) – To store a character into a char variable, you must enclose it within **SINGLE QUOTE** marks i.e. by ‘ ’.
- The double quotes are reserved for string (a collection of character).
- You can assign a char variable as an integer i.e. its integer value.
‘a’, ‘z’, ‘A’, ‘Z’, ‘?’, ‘@’, ‘0’, ‘9’

Declaration of char variable:

```
char c;   OR   char c, k, l;
```

Initialization of char variable:

```
char a='x';
```

Declaration and initializing can be done on separate lines.

```
char c;
```

```
c = 'a';
```

Printing value of char variable

- `printf( “%c”, a);`

Output: a

This causes the “ %c” to be replaced by value of character variable a.

- `printf(“\n %c”, a);`

Output: (blank space for new line)

a

“\n” is a new line character which will print the value of variable on newline.

Program using character constants

```
#include<stdio.h>

int main()
{
    char p, q, r, s, t, u;    /* declare char variables*/
    char m='e';              /* declare char variable*/
    p='W';                   /* initialize the rest */
    q='e';                   /* q=e is incorrect */
    r='l';                   /* s="c" is incorrect*/
    s='c';
    t='o';
    u= 109                   /* the ASCII value of m is 109 */
    printf("%c%c%c%c%c%c%c", p, q, r, s, t, u, m);
    return 0;
}
```

Output: Welcome

Integer Data Type

- Integer data types are allows to store numeric value in variables.
- Integer data type are represented as int.
- Size of int type variable is 2 or 4 byte depends on operating system configuration.
- If you try to store float value in integer variable then integer part will be stored in variable and the decimal part is ignored and the value assigned is rounded down from the actual value.
- Also assigning a character constant to an int variable assigns the ASCII value.

- Declaration on Integer variable:

```
int student_id;
```

- Here, student_id is a variable of type int.
- You can declare multiple variables at once in C programming. For example,

```
int student_id, marks;
```

- Input through scanf() and printing through printf() of integer variable will be done by %d.
- The size of int is usually 2 bytes (32 bit OS). And, it can take 2^{32} distinct states from -2147483648 to 2147483647

Program to print different variables.

```
#include<stdio.h>
main()
{   int a,b,c,d,e;
    a=5;
    b=8.2;
    c=5.3;
    d='A';
    e= 5.3 +8.2;
    printf("\n a=%d",a);
    printf("\n b=%d",b);
    printf("\n c=%d",c);
    printf("\n d=%d",d);
    printf("\n e=%d",e);
    printf("\n b+c=%d",b+c);
    return 0;
}
```

Output:

a=5
b=8
c=5
d=65
e=13
b+c=13

FLOATING POINT DATA TYPE:

- Floating point data type consists of 2 types. They are,

float

double

1. FLOAT:

- Float data type allows a variable to store decimal values.
- Storage size of float data type is 4. This also varies depend upon the processor in the CPU as “int” data type.
- We can use up-to 6 digits after decimal using float data type.
- For example, 10.456789 can be stored in a variable using float data type.

FLOATING POINT DATA TYPE:

- Floats are relatively easy to use but problems tend to occur when performing division

In general:

An int divided by an int returns an int.

An int divided by a float returns a float.

A float divided by an int returns a float.

A float divided by a float returns a float.

Declaration:

```
float f=3.4;
```

2. DOUBLE:

- Double data type is also same as float data type which allows up-to 15 digits after decimal.
- The range for double datatype is from $1\text{E}-37$ to $1\text{E}+37$.
- Declaration of double variable:
`double price;`

What is the difference between float and double:

- The size of float (single precision float data type) is 4 bytes. and the size of double (double precision float data type) is 8 bytes.
- Input using `scanf()` and printing using `printf()` of float data type will be done by `%f`.

int and float data types

```
#include<stdio.h>

main( )
{
    float x=8.0;
    float y= 8.00 + 7.00;
    printf("x=%f ",x);
    printf("y=%f ",y);
    return 0;
}
```

Output:

a=8.000000

b=15.000000

Here output is till 6 digit after decimal.

If you want to print till 3 digit the write %.3f
inplace of %f.

Changing the statement:

```
printf("x=%.3f" ,x);
```

```
printf("y=%.3f" ,y);
```

Output:

a=8.000

b=15.000

Summary

Basic Data Types

- **Integer (int):** Used to store whole numbers.
- **Floating point (float and double):** Used to store numbers with decimal points.
- **Character (char):** Used to store a single character.
- **Void:** Special data type with no associated value.

Integer Type (int)

Storage: Typically 4 bytes (depends on the system and compiler).

Range: -2,147,483,648 to 2,147,483,647 (for a 4-byte int).

Examples: `int age = 20;`

Floating Point Types (float and double)

float:

Storage: 4 bytes

Precision: 7 decimal places

double:

Storage: 8 bytes

Precision: 15 decimal places

Examples: `float pi = 3.14;` and `double precise Pi = 3.141592653589793;`

Character Type (char)

Storage: 1 byte

Range: -128 to 127 or 0 to 255

Usage: To represent single characters like 'A', '1', etc.
Examples: `char initial = 'A';`

The void Type

Meaning: "No Value"

Usage: Commonly used in function declarations to specify that the function doesn't return any value.

Example: `void printMessage();`

Derived Data Types

- **Introduction:** Derived from primary data types and built on their foundation.

Types:

- Array
- Pointer
- Structure
- Union

Type Modifiers

Introduction: Used to change the storage meaning of base data types.

Types:

signed
unsigned
long
short

Enumeration Data Type (enum)

Usage: Define a list of named integral constants.

Example:

```
enum days {SUN, MON, TUE, WED, THU, FRI, SAT};
```

User-defined Data Types

Introduction: Allow users to define their own data types.

Types:

Typedef
Structure
Union

- <https://theasciicode.com.ar/>
- <https://www.programiz.com/c-programming/c-data-types>
- <https://www.w3schools.in/c-tutorial/format-specifiers/>